

Tuple -

Container Data - List, Tuple, Set, Dictionary.

1) Tuple is one of the container data types in python.

2) We can create tuple using round brackets & elements are separated by comma.

eg. tuple1 = (10, 20, 30, 40, 50) ←
list1 = [10, 20, 30, 40, 50] ←

3) we can have homogenous or heterogeneous elements inside the tuple

eg. tuple2 = (10, 20, 30, 40, 50)

↓

homogeneous
tuple →

all elements are of int
data type.

tuple3 = (10, 10.1, 10+2j, True, "June")

heterogeneous →
tuple

↓
all elements are of
different data type

4) we can have duplicate elements
inside the tuple.

eg. tuple4 = (10, 20, 30, 10, 20)

5) we can create empty tuple

eg. tuple5 = ()

6) if we have only one element
inside our tuple

eg. tuple6 = (10,)

print(tuple6) → (10)
print(type(tuple6)) → <tuple>
tuple7 = (10)

print(tuple7) → 10

print(type(tuple7)) → <int>

• Accessing Tuple Elements -

1) We can print/access entire tuple using tuple name

eg. tuple 8 = (10, 20, 30, 40, 50)

print (tuple 8)

→ (10, 20, 30, 40, 50)

2) To access the individual elem. of tuple, we use indexing.

There are two types of indexing

1) Positive indexing - Starts from 0, Beg.

2) Negative indexing - Starts from -1, end

pos. ind	<u>0</u>	1	<u>2</u>	3	4
tuple 9	10	20	30	40	50
neg ind	-5	-4	-3	-2	-1

print (tuple 9) → (10, 20, 30, 40, 50)

print (tuple 9[0]) → 10

print (tuple 9[-5]) → 10

```

print (tuple9[1]) → 20
print (tuple9[-4]) → 20
print (tuple9[2]) → 30
print (tuple9[-3]) → 30
print (tuple9[3]) → 40
print (tuple9[-2]) → 40
print (tuple9[4]) → 50
print (tuple9[-1]) → 50

```

3. We can access the subtuple or group of elements of tuple using slicing.

4. Syntax for slicing

Syntax: tuple_name [start : end + 1 : step]

default values → 0 len(tuple) 1

Pos. indexing	0	1	2	3	4
tuple10	10	20	30	40	50

`print(tuple10)` $\rightarrow$ `(10, 20, 30, 40, 50)`
`print(tuple10[0:1:1])` $\rightarrow$ `(10)`
`print(tuple10[0:3:1])` $\rightarrow$ `(10, 20, 30)`
`print(tuple10[0:4:1])` $\rightarrow$ `(10, 20, 30, 40)`
`print(tuple10[1:4:1])` $\rightarrow$ `(20, 30, 40)`
`print(tuple10[0:5:1])` $\rightarrow$ `(10, 20, 30, 40, 50)`
`print(tuple10[: : :])` $\rightarrow$ `(10, 20, 30, 40, 50)`
`print(tuple10[: : 2])` $\rightarrow$ `(10, 30, 50)`

int - Basic data type

`age = 25` $\rightarrow$ int

`print(type(age))` $\rightarrow$ int

tuple - Container data type



`tuple1 = (10, 10.1, 10+2j, True, "gareeb")`

Looping in tuple -

we want to process each element of tuple, we have to iterate

tuple, & we perform this using while loop & for loop.

1] While loop :-

eg. tuple 11 = $\begin{matrix} & 0 & 1 & 2 \\ (10, 20, 30) \end{matrix}$

index = 0 (0 < 3)

```
while (index < len(tuple11)) :  
    print (tuple11[index])  
    index = index + 1
```

o/p-
10
20
30

2] for loop :

eg. tuple 12 = (10, 20, 30, 40, 50)

```
for var in tuple12 :  
    print (var)
```

o/p-
10
20
30
40
50

3] enumerate() function -

if we want to print index no. as well we use enumerate function.

eg. tuple13 = (10, 20, 30)

```
for index, value in enumerate(tuple13):  
    print(index, value)
```

o/p -

0	10
1	20
2	30

- Basic Tuple operations -

1] Immutability - Tuple is immutable, unchangeable.

once we defined / create a tuple we cannot add, remove, replace / modify elements of the tuple.

eg. tuple1 = (10, 20, 30)

```
tuple1[0] = 11  
print(tuple1)
```

→ Error
↓
Tuple is immutable.

2] Concatenation - we can join one tuple at the end of another

tuple using '+' operator.

eg. tuple1 = (10, 20, 30)

tuple2 = (40, 50, 60)

tuple3 = tuple1 + tuple2

(10, 20, 30) (40, 50, 60)

print (tuple3) → (10, 20, 30, 40, 50, 60)

3] Conversion - we can convert string, list, set, dictionary, range() into tuple.

we use tuple() function for this.

i) Converting String to tuple.

eg. str1 = "Hello"

tuple1 = tuple(str1)

print (tuple1) → ('H', 'e', 'l', 'l', 'o')

ii) Converting list to tuple

list1 = [10, 20, 30]

tuple2 = tuple(list1)

print (tuple2) → (10, 20, 30)

iii) Converting set into tuple

eg. Set1 = { 10, 20, 30 }

tuples = tuple(set1)

print(tuples) → (10, 20, 30)

iv) Converting dictionary into tuple

dict1 = { "name": "gareeb",

"age": 25 }

tuple4 = tuple(dict1)

print(tuple4) → ("name", "age")

v) Converting range() to tuple

eg. range1 = range(1, 11, 1)

tuple5 = tuple(range1)

print(tuple5) →

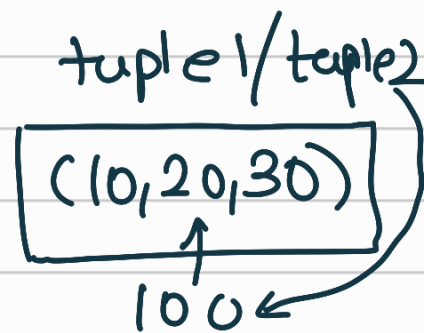
(1, 2, 3, 4, ..., 10)

5) Aliasing - Aliasing means multiple variables referring same tuple object in memory.

eg. tuple1 = (10, 20, 30)

tuple2 = tuple1

print(tuple2) → (10, 20, 30)



6. Searching - 'in' operator.

Using in operator we can check if particular element is present in the tuple or not

eg. tuple1 = (10, 10.1, True, 2+3j, "ganesha")

print(10 in tuple1) → True

print(100 in tuple1) → False

7. Identity - 'is' operator

Using identity operator we check if two variables refer to the same memory location or not.

eg. tuple1 = (10, 20, 30)

tuple2 = tuple1

print (tuple1 is tuple2) → True

8) Comparison -

We can compare contents of tuple

We compare it elementwise

eg. tuple1 = (10, 20, 30, 40)

tuple2 = (10, 20, 50, 60)

print (tuple1 < tuple2) → True

`print (tuple1 > tuple2) → False`

9] Emptiness - not operator

Using not operator, we can check if tuple is empty or not

eg. `tuple1 = ()`

`print(not tuple1) → True`

`tuple2 = (10, 20, 30)`

`print(not tuple2) → False`

- Built in functions in tuple

1. `len()` - It returns count of element in the tuple

eg. `tuple1 = (10, 20, 30)`

`print(len(tuple1)) → 3`

2. `max()` - It returns largest element in the tuple

eg. `tuple1 = (10, 20, 30)`

eg. tuple 2 = (10, 20, 30)

print (max (tuple2)) → 30

3] min() - It returns smallest element of the tuple

eg. tuple 1 = (10, 20, 30)

print (min (tuple1)) → 10

4. Sum() - It returns addition of all the elements of the tuple

eg. tuple 4 = (10, 20, 30)

print (Sum (tuple4)) → 60

5. any() - It returns true if any one element is true in the tuple.

eg. tuple 5 = (0, 0, 0, 1)

print (any (tuple5)) → True

tuple 5 = (0, 0, 0, 0)

`print(any(tuples)) → False`

6. `all()` - It returns true if all the elements of the tuple is true, if any one element is false it will return false.

eg, `tuple6 = (10, 20, 30, 40, 50)`

`print(all(tuple6)) → True`

`tuple6 = (10, 20, 30, 40, 0)`

`print(all(tuple6)) → False`

7. `sorted()` - It returns tuple in sorted (ascending) order.

`tuple1 = (50, 40, 10, 20, 30)`

`print(sorted(tuple1))`

↓ o/p - it returns list

`tuple(sorted(tuple1))`

```
print (tuple(sorted(tuple1)))
```

o/p - (10, 20, 30, 40, 50)

8. reversed() - It returns tuple in reversed order.

```
tuple8 = (10, 20, 30, 40, 50)
```

```
result = reversed(tuple1)
```

```
print (tuple(result))
```

o/p - (50, 40, 30, 20, 10)